

LISTA 03 –

1–10: Conceitos gerais de complexidade

1 -

Complexidade de tempo: quantidade de tempo para executar as operações.

Complexidade de espaço: quantidade de memória para executar as operações.

Exemplo em C onde o problema é a memória:

```
int matriz[10000][10000];
```

2 -

Levamos em consideração o termo dominante = $O(n^2)$

3 -

Busca Sequencial = $O(n)$

Busca Binária = $O(\log n)$

5 -

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < i; j++) {  
        cont++;  
    }  
}
```

Complexidade = $O(n^2)$

6 - Do menor para o maior:

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$

9 - Termo dominante: $O(n^3)$

11–20: Notação Big-O

11- Provar: $7n^3 + 4n^2 + 120 = O(n^3)$

Vamos mostrar que:

$$7n^3 + 4n^2 + 120 \leq c \cdot n^3$$

Para $n \geq 1$:

$$4n^2 \leq 4n^3$$

$$120 \leq 120n^3$$

Então:

$$7n^3 + 4n^2 + 120 \leq 7n^3 + 4n^3 + 120n^3 = 131n^3$$

Ou seja,

$$c = 131$$

Complexidade = $O(n^3)$

13-

4^n cresce mais rápido do que $O(3^n)$

14 –

a) $n^2 \log n = O(n^2)$

Falso ($\log n$ cresce)

b) $n \log n = \Omega(n)$

Verdadeiro

c) $\log_2 n = \Theta(\log_{10} n)$

Verdadeiro (mudança de base)

15 - Termo dominante: n^3

$$f(n) = n^3$$

21–25: Código em C

21 - Complexidade = $O(n^2)$

22 - Complexidade = $O(n)$

23 - Complexidade = $O(n^2)$

24 - Complexidade = $O(\log n)$

25 - Crescimento exponencial, logo complexidade = $O(2^n)$

26–30: Indução matemática + complexidade

$$T(n) = 2T(n/2) + 1$$

Base:

$$n = 1$$

$$2(1) - 1 = 1$$

Hipótese:

$$T(n/2) = n - 1$$

Indução:

$$T(n) = 2(n/2 - 1) + 1 = 2(n/2) - 2 + 1 = n - 1$$

$$T(n) = 2n - 1$$

28 - Base, Hipótese, Passo de Indução

$$1 + 3 + 5 + \dots + (2n - 1) = n^2$$

30 –

$$1 + 2 + \dots + n = [n(n+1)] / 2$$

Base: $n = 1$

$$1 = [1(1+1)] / 2$$

$$1 = 1$$

Hipótese: $n = k$

Indução: $n = k + 1$

$$(n + 1) (n/2 + 1) = (n + 1) (n+2)/2$$

31 - $T(n) = T(n-1) + 3n$

$$T(n) = 5 + 3(2 + 3 + \dots + n)$$

$$= 5 + 3 [(n(n + 1))/2 - 1]$$

$$T(n) = [(3n)(n + 1)]/2 + 2$$